5.2 PID 的设计与实现

在实际控制系统中，PID 设计涉及许多方面。我们从参数调整和实现方法来介绍。

PID 参数调整

PID 控制中涉及三个参数 k_p、k_i 和 k_d，分别对应比例、积分和微分控制的增益。如何选取这三个参数是 PID 设计的核心问题，通常需要根据系统的动态特性和性能指标进行调整。现有的准则和方法也有许多，包括经验调节法、Ziegler-Nichols 法、频率响应法以及现代控制理论下的优化方法等。

我们主要介绍从极点配置的角度进行 PID 参数调整的方法。

首先，回顾一下，系统的极点决定了系统的动态特性，包括响应速度、超调量和振荡频率等。对于一个 n 阶线性系统，

$$\dot{x}^n + a_{n-1}\dot{x}^{n-1} + \cdots + a_1\dot{x} + a_0 x = b_0 u$$

若它有 n 个极点 $\lambda_1, \lambda_2, \ldots, \lambda_n$，则系统的动态特性由这些极点的位置决定

$$x(t) = c_1 e^{\lambda_1 t} + c_2 e^{\lambda_2 t} + \cdots + c_n e^{\lambda_n t} = \sum_{i=1}^{n} c_i e^{\Re(\lambda_i)t} e^{j\Im(\lambda_i)t}$$

每一个极点 λ_i 的实部对应了 $c_i e^{\lambda_i t}$ 部分的衰减/增长速率，虚部对应了振荡频率。所以，如果控制器可以调整系统的极点位置，就可以间接地调节系统的动态特性，包括响应速度、超调量和振荡频率。

考虑受控系统的传递函数为 $P(s) = N(s)/D(s)$，PID 控制器为 $C(s) = k_p + \frac{k_i}{s} + k_d s$，则闭环系统的传递函数为：

$$G(s) = \frac{C(s)P(s)}{1 + C(s)P(s)} = \frac{C(s)N(s)}{D(s) + C(s)N(s)}$$

则闭环系统的极点由 $D(s) + C(s)N(s) = 0$ 决定。接下来，我们就可以通过调整 PID 控制器的参数 k_p、k_i 和 k_d 来改变闭环系统的极点位置，从而实现对系统动态特性的调节。一般情况下，我们希望极点位于左半平面，以保证系统的稳定性，并且希望

$$\begin{aligned} \mathrm{Re}(\lambda_i) < 0, \quad |\mathrm{Re}(\lambda_i)| \text{ 足够大} \quad &\Rightarrow \quad \text{足够快的响应速度} \\ |\mathrm{Im}(\lambda_i)| \text{ 不宜过大} \quad &\Rightarrow \quad \text{避免过度振荡} \end{aligned}$$

例 5.1. 考虑一个二阶系统 $P(s)=\frac{1}{s(s+a)}$，则单位负反馈闭环系统的特征方程为

$$s(s+a)+(k_p+\frac{k_i}{s}+k_d s)=0 \qquad \Rightarrow \qquad s^2(s+a)+(k_p s+k_i+k_d s^2)=0$$

如果，我们想设置三个闭环极点 $-p, -\sigma+i\omega$ 和 $-\sigma-i\omega$，则我们需要求解

$$s^2(s+a)+(k_p s+k_i+k_d s^2) \;=\; (s+p)(s+\sigma-i\omega)(s+\sigma+i\omega)$$

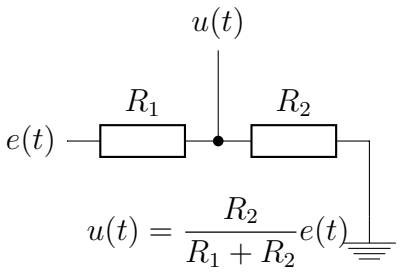
通过比较，我们可以得到

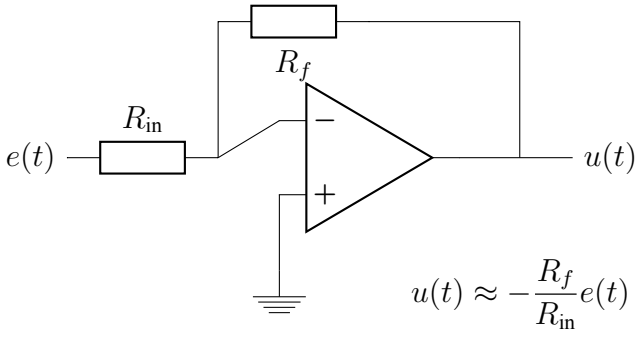
$$k_d=p+2\sigma-a \qquad k_p=2p\sigma+\sigma^2+\omega^2 \qquad k_i=p\left(\sigma^2+\omega^2\right)$$

PID 控制器的实现

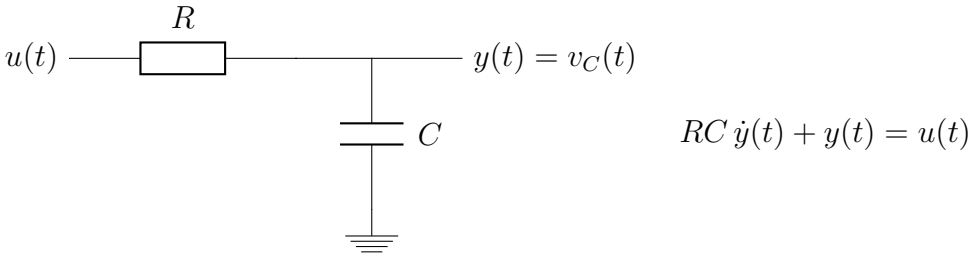
在当下数字控制的时代，我们可以通过计算机或芯片实现 PID 控制器的功能。但是这个代价是非常高的，尤其是我们需要在很多基础功能中实现 PID 控制时。在电路中，我们希望通过几个电阻、电容等简单原件就实现 PID 控制器的功能；在机械系统中，我们希望通过增加几个简单的弹簧和阻尼元件实现类似 PID 控制的功能；在生命系统中，我们也希望通过几个简单的生物机制实现类似 PID 控制的功能。

在实际系统中，我们可以较为容易地实现 P 控制和一阶系统。以电路系统为例，我们可以按照下面图所示的方式来实现比例控制。





针对一阶系统，我们可以通过下面的电路来实现。



那么，我们就可以利用比例控制和一阶系统的组合来实现 PI 控制器的功能。此

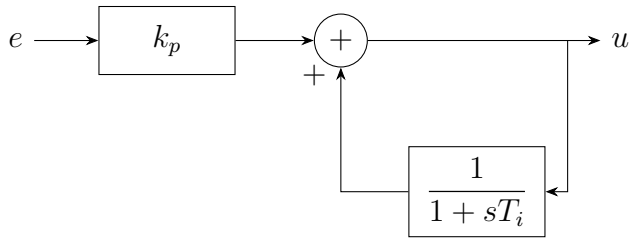


图 5.1: Automatic reset 结构 Åström et al. 2021, 图 11.3

时，闭环系统的传递函数满足

$$G(s) = \frac{U(s)}{E(s)} = k_p + \frac{k_p}{sT_i}$$

类似地，我们也可以通过组合的形式来实现微分控制器。此时，系统的传递函数

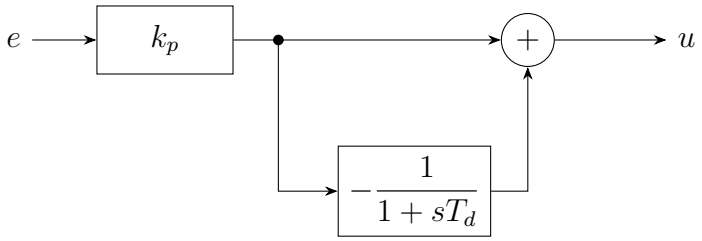


图 5.2: Filtered Derivative Åström et al. 2021, 图 11.3

满足

$$G(s) = \frac{U(s)}{E(s)} = k_p \frac{sT_d}{1+sT_d} \approx k_p T_d s \qquad \text{for } |sT_d| \ll 1$$

这个系统也称为滤波微分控制器（Filtered Derivative Controller）。它把让 $e(t)$ 中的低频率信号通过并计算了微分信号，而高频噪声则直接通过，不计算微分。若

$$e(t) = \sum_{n=-\infty}^{\infty} c_n e^{in\omega t}$$

则

$$u(t) = \sum_{n=-\infty}^{\infty} G(in\omega) c_n e^{in\omega t} \approx \sum_{|n|\omega T_d \ll 1} c_n k_p T_d (in\omega) e^{in\omega t} + \sum_{|n|\omega T_d \gg 1} k_p c_n e^{in\omega t}$$

积分器饱和（integrator Windup）

在实际控制系统中，执行器通常具有物理限制。例如，电机能够提供的最大力矩、汽车发动机能够提供的最大驱动力、阀门的最大开度等都是有限的。因此，控制输入通常满足

$$u_{\min} \le u(t) \le u_{\max}.$$

如果控制器计算得到的控制信号超过这一范围，实际作用在系统上的输入将发生饱和，即

$$u(t) = \text{sat}(u_c(t)) := \begin{cases} u_{\max}, & u_c(t) > u_{\max}, \\ u_c(t), & u_{\min} \le u_c(t) \le u_{\max}, \\ u_{\min}, & u_c(t) < u_{\min}, \end{cases}$$

其中 $u_c(t)$ 为控制器计算得到的控制信号，而 $u(t)$ 为实际施加在被控对象上的输入。

积分饱和与超调

考虑 PI 控制器 $u_c(t) = k_p e(t) + k_i \int_0^t e(\tau)\, d\tau$, 为了更加清楚地描述积分作用，引入积分状态

$$z(t) = \int_0^t e(\tau)\, d\tau, \qquad \dot{z}(t) = e(t),$$

则控制器可以写成

$$u_c(t) = k_p e(t) + k_i z(t).$$

当执行器发生饱和时，积分器可能继续累积误差，从而导致 $u_c(t)$ 远大于实际需要的控制输入。例如，当

$$u_c(t) > u_{\max}$$

时，实际控制输入只能取

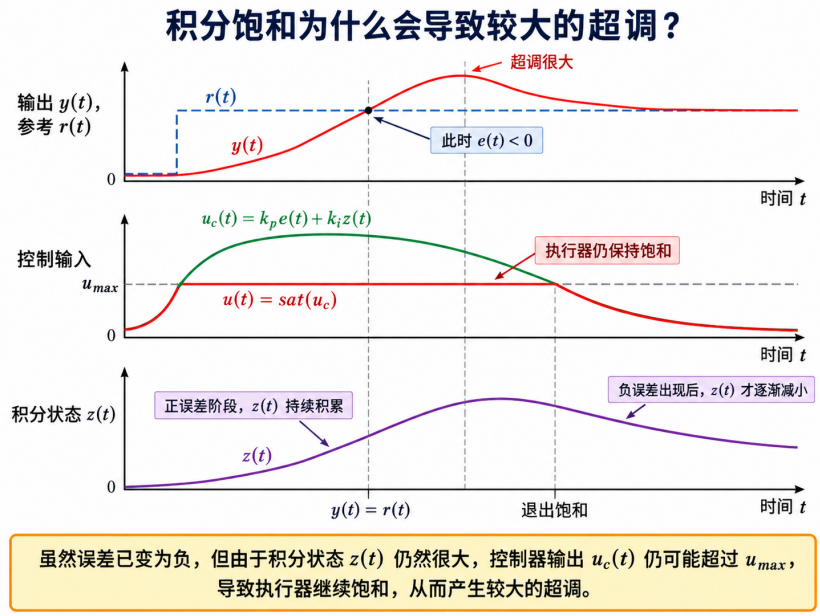
$$u(t) = u_{\max}.$$

此时，即使控制器继续增大 $u_c(t)$，实际作用在系统上的输入也不会继续增大。但是积分状态仍然继续累积，如果误差 $e(t)$ 在较长时间内保持为正，则 $z(t)$ 会持续增加，从而使 $u_c(t)$ 越来越大。这种现象称为

积分饱和（integrator windup）.

在实际系统中，假设参考输入突然增大，使得系统产生较大的正误差 $e(t) > 0$. 则上述积分饱和现象可能发生 (见下图)。随着系统输出逐渐接近参考值，误差 $e(t)$ 变小，甚至最终变成负值。但是此时 $z(t)$ 可能已经积累得很大，因此 $u_c(t) = k_p e(t) + k_i z(t)$

仍然可能大于 $u_{\max}$。因此，即使 $e(t)<0$, 执行器仍然可能保持在 $u(t)=u_{\max}$ 一段时间。只有当负误差持续足够长的时间，使积分状态 $z(t)$ 逐渐减小时，控制器才能退出饱和状态。这就会产生极大的超调。



抗积分饱和

为了避免上述问题，可以在控制器中加入抗积分饱和（anti-windup）机制。其基本思想是：当执行器已经饱和时，不应该继续让积分器沿着使饱和更加严重的方向累积。下面介绍两种常用方法。

条件积分最简单的方法是在执行器饱和时暂时停止积分。例如，可以定义

$$\dot{z}=\begin{cases} e, & u_{\min}<u_c<u_{\max} \\ 0, & u_c\geq u_{\max},\ e>0, \\ 0, & u_c\leq u_{\min},\ e<0, \\ e, & \text{其他情况}. \end{cases}$$

其含义是：

- 如果执行器没有饱和，则正常积分；
- 如果已经达到上限，而且 $e>0$ 会使控制器进一步增大，则停止积分；
- 如果已经达到下限，而且 $e<0$ 会使控制器进一步减小，则停止积分；

- 如果误差有助于控制器退出饱和，则允许继续积分。

反算抗饱和另一种常见方法称为反算（back-calculation），将积分器修改为

$$\dot{z}(t) = e(t) + k_{\text{aw}}\big(u(t) - u_c(t)\big),$$

其中，$k_{\text{aw}} > 0$ 为抗饱和增益。当没有发生饱和时 $u = u_c$，因此得到常规的积分控制器形式 $\dot{z} = e$。当发生上饱和时，

$$u = u_{\max} < u_c \qquad \Rightarrow \qquad \dot{z} = e + k_{\text{aw}}(u_{\max} - k_p e - k_i z)$$

其中有意向为 $-k_{\text{aw}} k_i z$，会快速地减小积分状态 z，从而防止积分器继续积累。反之，当发生下饱和时，第二项为正，会推动积分状态 z 增加，从而防止积分器继续积累。